1 Evaluation

Our evaluation asks whether historical GUI experience can be both *executable* and *transferable*. We organize the experiments around five questions. First, can a Function recorded once complete AndroidWorld tasks after device and layout changes? Second, can the same execution be replayed across B-MoCA environments without online planning? Third, how accurately and efficiently does OmniTransfer retrieve pages and map individual actions? Fourth, does the complete system remain effective under dynamic content, language changes, and interruptions on physical devices? Finally, how much latency, model usage, energy, and power does executable reuse remove from online execution? We additionally measure Function Store scaling, Checker behavior, and unrecovered failures.

The experiments deliberately separate full-agent performance from transfer quality. AndroidWorld and the physical-device workload exercise the complete system. B-MoCA disables planning to isolate layout generalization. OmniTransfer is evaluated separately at page and action granularity. This separation prevents a strong planner from hiding a weak replay mechanism, and prevents a replay failure from being misclassified as a reasoning failure.

1.1 Common Methodology

Cost accounting. Task success is computed over every task in the declared workload. Online cost is primarily averaged over all episodes that actually enter execution, including both successful and unsuccessful runs. A cell rejected during preflight is not counted as a zero-cost episode; instead, we report execution coverage separately. We also report success-only cost as a secondary measure of the resources required to finish one task. Each cost table therefore shows success rate and coverage beside model calls, tokens, and latency. Offline Function construction is a one-time cost and is never mixed with recurring online execution cost.

Repetition and uncertainty. AndroidWorld task outcomes use the official deterministic validator. Aggregate success rates are accompanied by 95% bootstrap confidence intervals over tasks. Component latency and physical-device measurements use at least five independent repeats after warm-up and report the mean, standard deviation, and P50/P95 where appropriate. Infrastructure failures are rerun and reported separately from method failures. All final tables state the number of executed episodes rather than presenting only percentages.

Mock figures. Figures in this draft are layout placeholders. Their values are illustrative, are visibly marked *MOCK—NOT RESULTS*, and are not used by any result sentence. Tables and result paragraphs use **TBD** until the corresponding experiment block is frozen. The plotting scripts later replace each mock from the CSV schema with no change to labels, captions, or section structure.

Table 1: AndroidWorld workload. Difficulty follows the official benchmark metadata rather than a partition derived from our results.

Role	Configuration	API/OS	Resolution	Tasks
Source	Small phone	API 33 / Android 13	720×1280	116
Target	Standard small phone	API 33 / Android 13	720×1280	116/method
Target	7.6in Foldable	API 34 / Android 14	1768×2208	116/method
Target	10.1in WXGA Tablet	API 33 / Android 13	1280×800	116/method
Official difficulty				61 / 36 / 19

1.2 AndroidWorld: Cross-Device Function Execution

Purpose. This experiment evaluates the main system claim: a successful execution collected on one device can become an executable Function that remains useful on another device. It jointly measures task completion and the online reasoning avoided by Function execution. The expected result is not merely a higher success rate; an effective reusable memory should preserve success while reducing model calls, token use, and end-to-end latency, with larger savings on longer tasks.

Workload and devices. We use all 116 tasks from the official AndroidWorld task list. We retain the benchmark's difficulty metadata: 61 Easy, 36 Medium, and 19 Hard tasks. The same metadata provides the official optimal-step estimate used in the task-length analysis. For each task, one successful source RunLog is collected with seed 111 and accepted only when the official validator succeeds and the state–action evidence is complete. Target executions use seed 113 and the official reset and validation procedures.

The source uses the canonical AndroidWorld phone configuration. The fixed main matrix uses a Standard Phone (Android 13, 720×1280) and Pixel Fold (Android 14, 2208×1840, unfolded). We additionally evaluate a Tablet (Android 13, 2560×1600, landscape) as a supplemental target. Together, the three targets stress resolution, aspect ratio, layout reflow, form factor, and OS-version changes. The fixed main matrix remains 116 tasks, five methods, and two targets, for 1,160 cells; Tablet results are reported separately and are not used to fill missing main-matrix cells.

Compared methods. **Fixed Replay** executes the recorded action sequence without model reasoning and exposes the brittleness of source selectors and coordinates. **MobileGPT** converts the shared source execution into its native task-local memory and adapts unresolved steps online. **AppAgent** converts the same source into its demonstration and UI documentation and plans from the current screen. **T3A** is the stock AndroidWorld agent

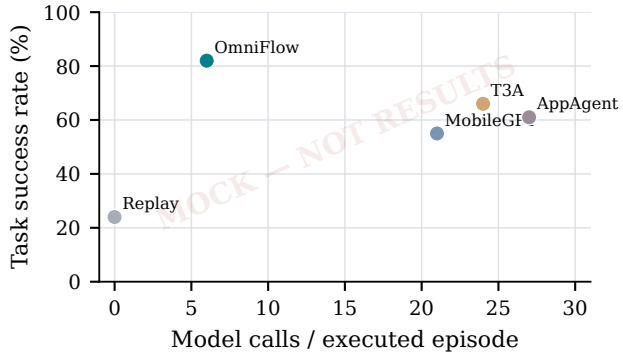


Figure 1: **Mock layout for AndroidWorld.** The final figure will place task success against model calls on the Standard Phone, Pixel Fold, and Tablet targets. Detailed difficulty and token results remain in Tables 2 and 3. Values shown here are not results.

supplied with a text-only description of the successful source execution; it still chooses every target action online. **OmniFlow** converts the RunLog into an executable Function, retrieves it from the current state and goal, and maps each recorded action using OmniTransfer. Mapping failures return to the planner and never replay source-device coordinates.

AutoDroid is evaluated as a supplemental UTG-based comparison on the subset of tasks supported by its available app and graph assets. It is not inserted into the fixed 1,160-cell matrix, and all methods are recomputed on exactly the same supported subset. This reports the value of UTG-level reuse without conflating missing UTG coverage with task failure.

Metrics. The primary metric is official task success rate (SR), reported by target and Easy/Medium/Hard difficulty. Efficiency metrics are model calls, prompt and output tokens, end-to-end latency, physical GUI actions, Function-served actions, planner actions, and fallback steps. Coverage is the fraction of scheduled cells that enter execution. We additionally regress calls, tokens, and latency against official optimal steps to determine whether the savings grow with task length.

Execution-time decomposition. To retain the detailed cost accounting, we separately report time spent in the model service, physical GUI I/O, and one-time memory preparation. This prevents a lower total from being attributed to Function execution when it instead comes from a different observation path or an incomplete preparation stage.

Historical-experience recall. We also evaluate whether a compatible source memory is retrieved before the agent generates a new action. Each query contains a target-device state and goal with a known compatible source execution. Recall@1 measures immediate fast path availability; Recall@3 and Recall@5 measure whether the correct experience appears in a small candidate set. Fixed Replay, AppAgent, and T3A do not expose a comparable ranked retrieval operation and remain only in the end-to-end comparison.

Supplemental UTG subset. AutoDroid's public assets cover only a subset of AndroidWorld. We therefore report its coverage and recompute every method on exactly this subset rather than adding unsupported cells to the main matrix.

Results. OmniFlow completes **TBD/TBD/TBD**% of Easy/Medium/Hard tasks on the Standard Phone, **TBD/TBD/TBD**% on Pixel Fold, and **TBD/TBD/TBD**% on the supplemental Tablet. The strongest published baseline obtains **TBD/TBD/TBD**%, **TBD/TBD/TBD**%, and **TBD/TBD/TBD**%, respectively. Across all executed episodes, OmniFlow reduces model calls from **TBD** to **TBD**, tokens from **TBD** to **TBD**, and latency from **TBD** to **TBD** seconds while changing overall SR from **TBD**% to **TBD**%. On Hard tasks, the call reduction is **TBD**%, showing **TBD** about whether savings grow with task length. The supplemental AutoDroid subset contains **TBD** tasks; OmniFlow changes complete success from **TBD**% to **TBD**% and online calls from **TBD** to **TBD** on this common subset.

1.3 B-MoCA: Replay-Only Layout Generalization

Purpose. AndroidWorld measures the complete system but cannot reveal whether a successful run came from transfer or from fallback planning. B-MoCA therefore isolates the transferable-execution claim. Given a successful source trace, each method must relocate and execute its recorded actions in another environment. Online model calls, newly generated actions, and planner fallback are disabled. This experiment measures layout generalization rather than general agent accuracy.

Dataset and protocol. The current aligned corpus contains 77 tasks and 226 officially successful traces: 75 source traces and 151 target traces. It provides 221 cross-environment trace alignments, 1,258 aligned page pairs, and 726 aligned action pairs. The replay protocol uses environment 100 as the source and environments 101 and 105 as held-out targets. Environment 105 produces the larger layout shift and is reported separately. These corpus statistics describe available evidence; they are not counted as online replay successes.

We compare normalized-coordinate replay, exact UTG identity, structured UTG attribute matching, and OmniTransfer. A step is correct when the predicted action targets the reviewed target element. An episode succeeds only when every replayed step is correct and the official target validator succeeds. We report step accuracy,

Table 2: AndroidWorld task success by official difficulty. Each cell will report SR and the number of successful tasks. Coverage is reported independently so unavailable runs cannot appear as inexpensive failures. Standard Phone and Pixel Fold form the fixed main matrix; Tablet is reported as a supplemental target.

Method	Target	Easy SR	Medium SR	Hard SR	Overall SR	Coverage
Fixed Replay	Standard Phone	TBD	TBD	TBD	TBD	TBD/116
	Pixel Fold	TBD	TBD	TBD	TBD	TBD/116
	Tablet	TBD	TBD	TBD	TBD	TBD/116
MobileGPT	Standard Phone	TBD	TBD	TBD	TBD	TBD/116
	Pixel Fold	TBD	TBD	TBD	TBD	TBD/116
	Tablet	TBD	TBD	TBD	TBD	TBD/116
AppAgent	Standard Phone	TBD	TBD	TBD	TBD	TBD/116
	Pixel Fold	TBD	TBD	TBD	TBD	TBD/116
	Tablet	TBD	TBD	TBD	TBD	TBD/116
T3A	Standard Phone	TBD	TBD	TBD	TBD	TBD/116
	Pixel Fold	TBD	TBD	TBD	TBD	TBD/116
	Tablet	TBD	TBD	TBD	TBD	TBD/116
OmniFlow	Standard Phone	TBD	TBD	TBD	TBD	TBD/116
	Pixel Fold	TBD	TBD	TBD	TBD	TBD/116
	Tablet	TBD	TBD	TBD	TBD	TBD/116

Table 3: AndroidWorld online cost. The primary columns average all executed episodes, including failures. Success-only values are secondary.

Method	Calls	Prompt tok.	Output tok.	Latency s	GUI actions	Reused actions	Planner actions	Fallbacks	Success-only latency
Fixed Replay	0	0	0	TBD	TBD	TBD	0	0	TBD
MobileGPT	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
AppAgent	TBD	TBD	TBD	TBD	TBD	0	TBD	TBD	TBD
T3A	TBD	TBD	TBD	TBD	TBD	0	TBD	TBD	TBD
OmniFlow	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 4: AndroidWorld latency decomposition. Online columns are averaged over executed episodes; preparation is paid once per source task.

Method	Model s	GUI I/O s	Total s	Prep. s/task
Fixed Replay	0	TBD	TBD	TBD
MobileGPT	TBD	TBD	TBD	TBD
AppAgent	TBD	TBD	TBD	TBD
T3A	TBD	TBD	TBD	TBD
OmniFlow	TBD	TBD	TBD	TBD

Table 5: Historical-experience retrieval on target-device states.

Retrieval method	R@1	R@3	R@5	Query ms
MobileGPT embedding	TBD	TBD	TBD	TBD
AutoDroid UTG	TBD	TBD	TBD	TBD
OmniFlow state+goal	TBD	TBD	TBD	TBD

complete replay rate, unresolved-step rate, and wrong-action rate. Every row must have `model_calls=0` and `fallback_steps=0`.

Table 6: Common subset supported by AutoDroid's UTG assets.

Method	Tasks	Small SR	Fold SR	Calls	Time s
Fixed Replay	TBD	TBD	TBD	0	TBD
MobileGPT	TBD	TBD	TBD	TBD	TBD
AppAgent	TBD	TBD	TBD	TBD	TBD
T3A	TBD	TBD	TBD	TBD	TBD
AutoDroid	TBD	TBD	TBD	TBD	TBD
OmniFlow	TBD	TBD	TBD	TBD	TBD

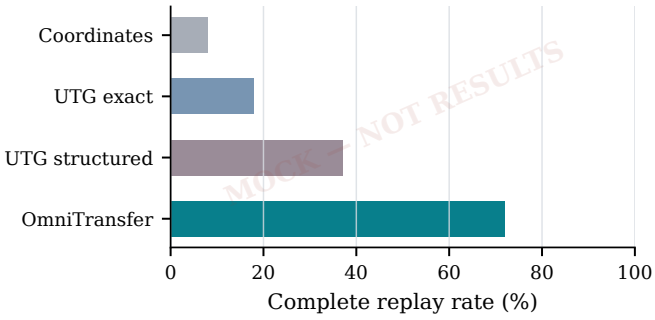


Figure 2: **Mock layout for planning-free B-MoCA replay.** The final figure will compare complete replay; step-level errors and environments are reported in Table 7. Values shown here are illustrative.

Results. OmniTransfer correctly maps **TBD/TBD** steps and completes **TBD/TBD**% of episodes in envi-

Table 7: Planning-free replay on B-MoCA. No method can repair a replay with online reasoning.

Method	Target	Step acc.	Complete	Unresolved	Wrong
Normalized coordinates	env101	TBD	TBD	TBD	TBD
	env105	TBD	TBD	TBD	TBD
UTG exact identity	env101	TBD	TBD	TBD	TBD
	env105	TBD	TBD	TBD	TBD
UTG structured match	env101	TBD	TBD	TBD	TBD
	env105	TBD	TBD	TBD	TBD
OmniTransfer	env101	TBD	TBD	TBD	TBD
	env105	TBD	TBD	TBD	TBD

ronments 101/105, compared with **TBD/TBD**% for the strongest UTG baseline. It rejects **TBD**% of steps and maps an incorrect target in **TBD**%. Because no planner is available, this difference directly measures the value of step-level learned mapping over graph-level identity and attribute reuse.

1.4 OmniTransfer: Page Recall and Step Matching

Purpose. OmniTransfer is the mechanism that makes an executable Function transferable. It serves two related operations with a lightweight learned UI representation. Page recall asks which historical Function state is compatible with the current state; step matching asks where a recorded action should execute on the current page. Evaluating the two operations separately distinguishes retrieval errors from action-localization errors.

Datasets and baselines. We use two frozen, separately scored protocols. ASE supplies reviewed cross-page node correspondences and is the primary step-matching benchmark; its test split contains 98 page pairs, 196 directions, and 1,286 positive node correspondences. PAPT supplies verified phone–tablet page pairs and is used only for page retrieval. Following the earlier OmniTransfer diagnostic, we use the deterministic PAPT-128 subset: 128 pairs from 15 apps, disjoint from the 418 PAPT training apps. Each query ranks the target pages from its known app (33.77 candidates on average, at most 61), rather than a global gallery. PAPT therefore measures page compatibility, not node-level supervision. Only reviewed ASE correspondences contribute to mapping accuracy.

For ASE we compare three measured baselines: exact equality of normalized text/content-description fields, exact resource-id equality (a diagnostic only, since resource IDs are not model inputs), and the previous clean-v9 learned matcher. LightGlue and SuperGlue are useful architectural references for iterative correspondence, but they are keypoint/one-to-one matchers and have no directly comparable XML-node evaluation under this protocol; we do not report fabricated numbers for them.

Page recall. For each phone query, the PAPT-128 gallery contains the historical tablet states from the same app. We report Recall@1, Recall@3, Recall@5, Top-2 recovery, encoding latency, request count, and cost. Recall@1 is primary because the online fast path needs an immediately executable page. The OmniTransfer representation is computed locally from XML, visual evidence, and sparse local relations; it makes zero API requests and uploads neither modality. The PAPT-128 result is a representation-level component evaluation, not a claim of end-to-end planner success.

Table 8: Datasets used by the OmniTransfer evaluation. Page pairs without reviewed node labels are not counted as step-matching supervision.

Dataset	Current scale	Evaluation role
ASE	98 test pairs / 1,286 links	Step matching
PAPT-128	128 pairs / 15 apps	Page recall

Step matching. Each query contains a source state, its recorded action target, and a target state with all candidate nodes. We report Top-1 accuracy, Recall@3, Recall@5, wrong-action rate after thresholding, and CPU P50/P95 latency. A missing or low-confidence target is an explicit transfer failure. The runtime never passes through source coordinates, node IDs, or resource IDs as a fallback.

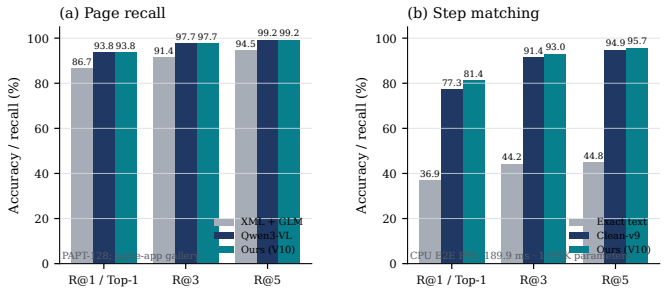


Figure 3: Measured OmniTransfer component results. The left panel compares PAPT-128 page recall against compressed-XML and screenshot embeddings; the right panel compares ASE step matching against exact-text and the previous clean-v9 matcher. Higher bars are better.

Frozen step-matching result. The current frozen test contains 98 page pairs, 196 mapping directions, and 1,286 positive correspondences. OmniTransfer obtains 81.42% Top-1 accuracy, 93.00% Recall@3, and 95.72% Recall@5. It improves the exact-text baseline by 44.56 percentage points and the previous clean-v9 matcher by 4.12 points at Top-1. On CPU, warm end-to-end mapping takes 88.5 ms at P50 and 189.9 ms at P95. The checkpoint contains 1,194,582 parameters and uses no remote service. As a fail-closed diagnostic, a dev-calibrated high-confidence gate (top assignment probability $\tau = 0.9999$) accepts 280/1,286 test rows (21.8% coverage) and makes one wrong accepted action (0.36% of accepted rows, 0.08% of all rows); this

Table 9: OmniTransfer component results under the two frozen protocols. Page-recall latency is mean/P95 cold encoding per page; step-matching latency is warm CPU end-to-end P50/P95. API usage and list-price cost are zero for local or exact baselines; the two remote embedding rows report their measured request totals and list prices.

Task	Dataset/method	R@1/Top-1	R@3	R@5	Latency P50/mean ms	P95 ms	Params	API tokens / cost
Page recall	PAPT-128, compressed XML + GLM-Embedding-2	86.72	91.41	94.53	214.1	1309.3	–	95,685 / CNY 0.0478
Page recall	PAPT-128, Qwen3-VL screenshot	93.75	97.66	99.22	1004.2	1489.2	–	322,944 / CNY 0.5751
Page recall	PAPT-128, OmniTransfer V10-K16	**93.75**	**97.66**	**99.22**	**33.0**	**63.7**	**1,194,582**	**0 / CNY 0**
Step match	ASE, exact text/content-desc	36.86	44.25	44.79	–	–	–	0 / CNY 0
Step match	ASE, exact resource-id	0.00	0.00	0.00	–	–	–	0 / CNY 0
Step match	ASE, clean-v9 matcher	77.29	91.37	94.87	–	–	761,883	0 / CNY 0
Step match	ASE, OmniTransfer V10-K16	**81.42**	**93.00**	**95.72**	**88.5**	**189.9**	**1,194,582**	**0 / CNY 0**

Table 10: Measured V10 ablations on the frozen ASE mapping split. Wrong action is only defined for the separately reported reject-gate diagnostic.

Configuration	Top-1	R@3	R@5	P95 ms
Full model	**81.42**	**93.00**	**95.72**	**189.9**
w/o local UI relations	74.65	89.58	93.16	261.8
w/o visual features	73.64	87.33	92.15	390.0
direct node descriptor only	56.92	69.60	76.59	–

reject-rate trade-off is reported separately from ranking accuracy and is not used to inflate Top-1.

Ablations. The ASE ablations test whether the gain comes from the learned local graph and the multimodal node state rather than from a late exact-text shortcut. The released V10 checkpoint has no separately parameterized cross-page-feedback module, so a "without bidirectional association" row would not be a valid ablation and is omitted.

Page-recall result and conclusion. On PAPT-128, OmniTransfer V10-K16 reaches 93.75% Top-1 and 99.22% R@5, matching Qwen3-VL's 120/128 Top-1 hits and exceeding compressed XML by 7.03 percentage points. Its Top-2 recall is 96.88% (+3.12 points over Top-1), with a mean Top-1–Top-2 cosine gap of 0.270. The local encoder averages 33.0 ms per page (63.7 ms P95), compared with 214.1 ms for compressed XML and 1,004.2 ms for Qwen3-VL. The corresponding PAPT-128 costs are CNY 0.0478 and CNY 0.5751 for the two remote baselines, versus zero API requests and zero recurring API cost for OmniTransfer.

Taken together, the two experiments separate the roles of the representation. On the requested PAPT-128 protocol, page recall matches the cloud vision embedding while remaining local and inexpensive. Step matching is the stronger structural claim: the sparse local graph and cross-page correspondence raise node Top-1 from 56.92% for direct descriptors to 81.42%. The conclusion is deliberately bounded to these frozen protocols; OmniTransfer does not claim to dominate every visual embedding on every gallery. It provides a single executable, fail-closed transfer mechanism whose page selection costs no API calls and whose action localization uses explicit UI-node structure, local neighbors, and multimodal evidence.

1.5 Physical Devices and Dynamic Tasks

Purpose. Benchmark emulators provide controlled layouts but do not capture personalized feeds, WebViews, advertisements, language settings, permission dialogs, and vendor-specific UI changes. This experiment evaluates whether Functions can be recalled and executed under these dynamic conditions on physical devices. More importantly, it tests reuse inside open-ended automation: a Function learned from a short recurring task must remain useful when a later task has a different semantic goal, aggregates multiple results, or invokes the same capability several times. It also tests whether the complete system can remove transient interruptions and resume the same Function step.

Applications and tasks. The workload contains 30 source–target task pairs, five each from Meituan, Xiaohongshu, Alipay, Ctrip, Douyin, and WeChat. These applications cover local services, user-generated content, super-app services, travel, short video, and communication, and expose native views, WebViews, mini-programs, personalized feeds, advertisements, and transient overlays. We choose common automation needs—content collection, repeated purchase preparation, constrained search, travel planning, service access, and message or file handling—rather than fixed commands with predetermined UI targets.

Each pair is intentionally asymmetric. The *source task* is a successful, short recurring operation from which OmniFlow constructs Functions. The *target task* expresses a new and more complex user goal that must compose or repeatedly invoke those Functions while the host agent handles task-specific reasoning. For example, the source task in Xiaohongshu asks the agent to find and save a beginner-friendly mobile-photography composition tutorial. The target task asks it to prepare for a trip to Shanghai by finding three informative posts—one each for food, accommodation, and transportation—and saving all three. The target therefore reuses content search, result inspection, post opening, and saving several times, but neither the selected posts nor the complete execution path are fixed by the source. Other task pairs similarly progress from one item to a constrained collection, from one service to a multi-service workflow, or from one retrieval to multi-result aggregation. The complete prompts, preconditions, semantic constraints, and reset procedures are listed in

the appendix.

Cross-device protocol. Pixel 9 Pro is fixed as the source device. OnePlus 12, vivo X300 Pro, and HONOR Magic V5 are held-out targets; the source is not selected separately for each target. We execute the source member of each pair once, accept it only after successful completion and complete RunLog capture, and construct its Functions before exposing the target member. Every target task is then executed on each held-out device. A target run may select different merchants, posts, videos, hotels, or services from the source run; success depends on satisfying the task-level constraints, not on reproducing the recorded trajectory. For the Xiaohongshu example, the oracle requires three distinct Shanghai-related saved posts covering food, accommodation, and transportation. We verify structural outcomes from the final app state and manually audit semantic constraints that cannot be determined from the UI tree alone.

Each target is executed **TBD** times after **TBD** warm-up runs under matched account, network, app-version, and initial-state conditions. Runs are additionally tagged by layout reflow, language change, dynamic or personalized content, and popup/ad interruption. Tasks that involve shopping, payment, booking, public posting, or other consequential actions terminate before final confirmation, and reversible state such as likes, saves, carts, and test messages is reset between runs.

The Checker is generated from RunLogs but is independent of Function recall and action mapping. At each observation it may detect a registered transient state, execute the corresponding dismissal action, and resume the same Function step. We compare OmniFlow with and without the Checker, while keeping the Function Store and transfer model fixed.

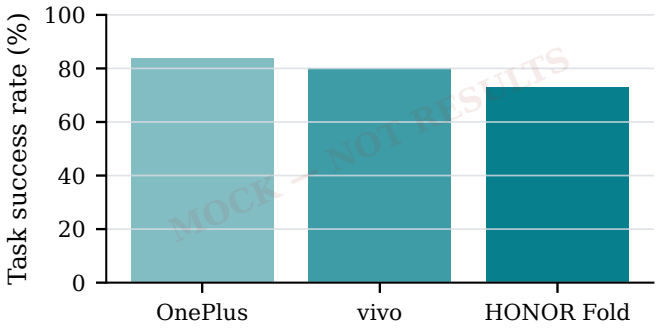


Figure 4: **Mock layout for real-device dynamic tasks.** The final figure will report success by held-out device; dynamic UI conditions remain in Table 11. Values shown here are illustrative.

Results. Across the three held-out devices, OmniFlow completes **TBD/TBD/TBD**% of tasks and serves **TBD/TBD/TBD**% of actions from Functions. Layout, language, dynamic-content, and popup/ad subsets obtain **TBD**, **TBD**, **TBD**, and **TBD**% SR. Enabling the Checker detects **TBD/TBD** interruptions, resumes **TBD**% of detected cases, and falsely activates on **TBD**% of normal states. These results will determine whether cross-device reuse survives the UI changes that are not represented by a fixed emulator snapshot.

1.6 Latency, Model Usage, Energy, and Power

Purpose. Executable reuse is valuable only if its local retrieval and mapping cost is smaller than the online reasoning it replaces. We therefore compare the same agent in two modes: step-by-step execution disables Functions and invokes the model for every decision, while Function-augmented execution enables OmniFlow without changing the model, observation format, physical action API, or success criterion. This paired design attributes differences to reusable execution rather than to a different planner or deployment split.

Measurement protocol. For each physical device, we record end-to-end task latency, model calls, prompt/output tokens, active energy, mean active power, peak power, peak memory, and device temperature. Active energy integrates power above the device's idle baseline over the complete executed episode. Cold and warm page encoding are reported separately. The Function construction and RunLog collection costs are one-time costs and are reported outside the online total.

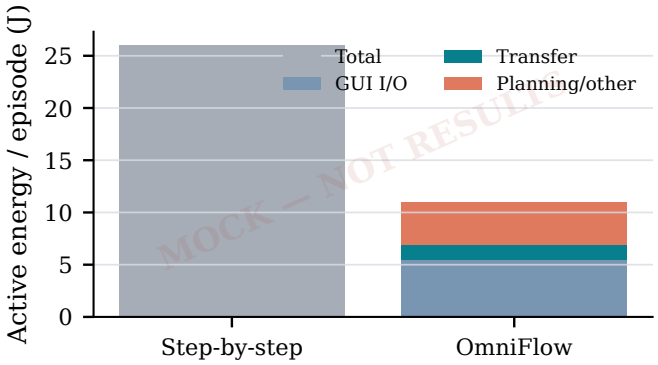


Figure 5: **Mock layout for resource cost.** The final figure will compare active energy per executed episode and attribute OmniFlow's stacked cost. Values shown here are illustrative.

Results. OmniFlow changes active energy from **TBD** to **TBD** J, latency from **TBD** to **TBD** s, and model calls from **TBD** to **TBD** per executed episode, while changing SR from **TBD**% to **TBD**%. OmniTransfer consumes **TBD** J, or **TBD**% of OmniFlow's active energy; GUI observation and input consume **TBD** J, and remaining planner fallbacks consume **TBD** J. Success-only energy is

Table 11: Execution of the 30 complex target tasks on held-out physical devices. A Function hit denotes at least one recalled Function; mapped steps count actions served after target-device relocation.

Target	Condition	Tasks	SR	Function hit	Mapped steps	Fallbacks	Calls	Tokens	Latency s
OnePlus 12	All	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
vivo X300 Pro	All	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
HONOR Magic V5	All	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
All targets	Layout reflow	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
All targets	Language change	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
All targets	Dynamic content	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
All targets	Popup/ad	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 12: End-to-end online resource cost. Means include successful and unsuccessful executed episodes; success-only cost is reported separately.

Mode	SR	Coverage	Latency s	Active J	Mean W	Peak W	Calls	Tokens	Peak MB	Success-only J
Step-by-step agent	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
OmniFlow	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD

TBD J versus **TBD** J, confirming **TBD** after controlling for early termination.

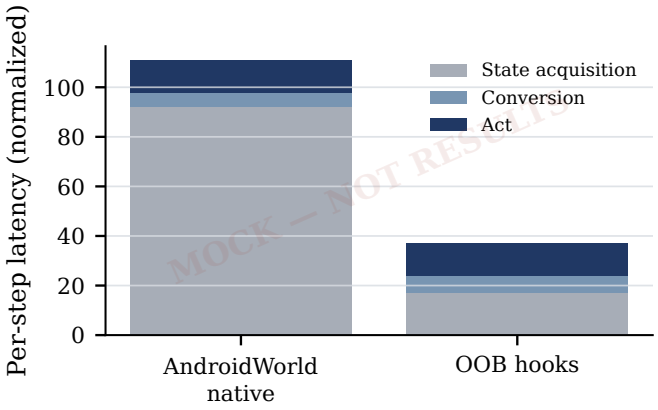


Figure 6: **Mock layout for observation/action decomposition.** The final stacked bars will report state acquisition, conversion, and action latency. Mock values are not results.

Observation/action decomposition. To isolate the source of the control-plane speedup, we measure observation and action latency separately on the same AndroidWorld replay configuration, with UI stabilization disabled (`stable=false`). We replay `AudioRecorderRecordAudio` on `small5554` in five independent runs and report the mean across runs. AndroidWorld's native observation path takes 1334 ± 32 ms per step, including 1231 ms in native `get_state`. Screenshot capture, XML conversion, and image encoding together account for only approximately 76 ms. OOB reduces state acquisition to approximately 207 ms while leaving the host-side conversion cost comparable. The speedup therefore comes primarily from removing repeated host–device coordination during state acquisition; it does not change the planner, action semantics, or final observation format. The `observe+act` value is the sum of the independently measured mean observation and action latencies and is reported only as a decomposition, not as a jointly timed confidence interval.

1.7 Scalability, Checker Safety, and Failures

Function Store scaling. A reusable system must support more Functions without turning every observation into an expensive global scan. We construct stores containing 10, 100, 1,000, and 10,000 Functions, with multiple records allowed for the same semantic Function. For held-out state–goal queries, we measure Recall@1, P50/P95 query latency, index size, and peak memory. This experiment tests whether aggregating additional device and layout variants improves recall without prohibitive lookup cost.

Table 13: First unrecovered OmniFlow failure.

Failure source	Easy	Medium	Hard	Total
Function recall miss	TBD	TBD	TBD	TBD
Transfer rejected	TBD	TBD	TBD	TBD
Wrong action mapped	TBD	TBD	TBD	TBD
Checker failure	TBD	TBD	TBD	TBD
Planner fallback failure	TBD	TBD	TBD	TBD
Environment failure	TBD	TBD	TBD	TBD

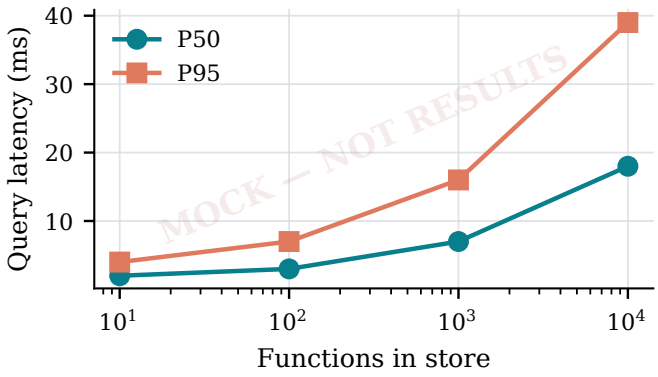


Figure 7: **Mock layout for Function Store scaling.** The final figure will show P50/P95 retrieval latency as store size grows; Recall@1 and memory remain in the result paragraph. Values shown here are illustrative.

Checker and failure attribution. Checker evaluation uses interrupted and normal states not used to construct the corresponding rule. We report interruption detection, successful same-step resume, false activation, and added observation latency. For end-to-end failures, each run is assigned to its first unrecovered cause: Function recall miss, transfer rejection, wrong mapped action, Checker failure, planner failure, or environment failure. Transfer rejection and wrong mapping are separated because the former is a safe miss that only adds fallback cost, whereas the latter can reduce task success.



Figure 8: **Mock layout for safety and failure analysis.** The final figure reports independent Checker detection, resume, and false activation; failure attribution remains in Table 13.

Results. At 10,000 Functions, Function recall takes **TBD/TBD** ms at P50/P95, uses **TBD** MB of index memory, and retains **TBD**% Recall@1. Adding multiple records per semantic Function changes Recall@1 from **TBD**% to **TBD**%. Across end-to-end failures, **TBD**% are safe recall or transfer rejections, **TBD**% are incorrect mapped actions, and **TBD**% arise after planner fallback. The Checker adds **TBD** ms per observation and changes interrupted-task success from **TBD**% to **TBD**%.